

Lab Report: Week 5 Session B Lab 2

SCS-CN Runoff Modeling

Mahmudul Hasan (Student ID: 4125999049)

May 21, 2026

Abstract

This report documents Week 5 Session B Lab 2: implementation of the **SCS-CN** direct runoff method in Python, parameter sensitivity over curve numbers $CN \in \{60, 70, 80, 90, 95\}$ and rainfall $P \in [0, 100]$ mm, and domain validation ($Q \leq P$, $Q = 0$ when $P \leq I_a$, higher $CN \Rightarrow$ more runoff). The function `calculate_runoff(P, CN)` reuses the Week 3 Session B TDD implementation in `week5_session_b_lab2/`. All **7** `pytest` cases passed; `sensitivity_analysis.py` saved `figures/runoff_sensitivity.png` and passed physical checks; reference cases $Q(50, 80) \approx 13.80$ mm and $Q(80, 85) \approx 43.55$ mm match course expectations. Evidence: three terminal screenshots and the sensitivity figure; source in Appendix A–E.

1 Introduction

The Soil Conservation Service Curve Number (SCS-CN) method estimates rainfall excess (direct runoff) from total storm depth and a land-use-dependent curve number. This lab translates the metric formulas into tested Python code, explores sensitivity of Q to CN , and verifies hydrologic plausibility—skills that complement Week 5 Session A Lab 1 (live rainfall from an API) with **process-based** runoff depth in millimetres.

2 Objectives

- Implement `calculate_runoff(P, CN)` with boundary conditions and validation.
- Run comprehensive `pytest` edge cases (Exercise 1).
- Produce sensitivity plots and a pandas DataFrame (Exercise 2).
- Validate $Q \leq P$, zero runoff below I_a , and CN ordering (Exercise 3).
- Document work in `prompt_log.md` (Exercise 4).

3 Methods

3.1 SCS-CN formulation (Exercise 1)

All depths in **millimetres (mm)**:

$$S = \frac{25400}{CN} - 254, \quad I_a = 0.2 S,$$
$$Q = \begin{cases} 0 & \text{if } P \leq I_a, \\ \frac{(P - I_a)^2}{P - I_a + S} & \text{otherwise,} \end{cases}$$

with Q clipped to $[0, P]$. CN is validated in $[1, 100]$ to avoid division by zero on S . Implementation: `src/runoff.py` (developed in Week 3 Session B via TDD; reused here).

3.2 Tests (Exercise 1)

`tests/test_runoff.py` — seven cases: normal ($P=50$, $CN=80$), $P=0$, $P < I_a$, $CN=100$ (impervious), negative P , invalid CN , and grid check $Q \leq P$.

3.3 Sensitivity analysis (Exercise 2)

`sensitivity_analysis.py` builds a wide DataFrame (`rainfall_mm`, `Q_CN60`, ...), saves `figures/sensitivity_` plots rainfall vs. Q for each CN with a dashed $Q = P$ upper bound, and runs embedded domain checks.

3.4 Reference validation (Exercise 3)

`validate_reference.py` spot-checks course reference points; `sensitivity_analysis.domain_checks()` asserts $Q \leq P$ and monotonic increase of Q with CN at $P=50$ mm.

4 Results

4.1 File inventory

Table 1: Submitted files in `week5_session_b_lab2/`.

File	Role
<code>src/runoff.py</code>	SCS-CN <code>calculate_runoff</code>
<code>tests/test_runoff.py</code>	pytest suite (7 tests)
<code>sensitivity_analysis.py</code>	DataFrame, plot, domain checks
<code>validate_reference.py</code>	Hand-check reference cases
<code>figures/runoff_sensitivity.png</code>	Sensitivity figure
<code>prompt_log.md</code> , <code>prompt_*.txt</code>	Documentation

4.2 Terminal verification

Figures 1a–1c show the three commands required for submission evidence.

```

mahmudul-robotics@robotics-ai:~/Downloads/Software Development/ai_water_lab/week5_session_b_lab2$ cd ~/home/mahmudul-robotics/Downloads/Software Development/ai_water_lab/week5_session_b_lab2
mahmudul-robotics@robotics-ai:~/Downloads/Software Development/ai_water_lab/week5_session_b_lab2$ python3 -m pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/mahmudul-robotics/Downloads/Software Development/ai_water_lab/week5_session_b_lab2
plugins: anyio-4.5.2
collected 7 items

tests/test_runoff.py::test_normal_case PASSED [ 14%]
tests/test_runoff.py::test_no_rainfall PASSED [ 28%]
tests/test_runoff.py::test_rainfall_less_than_initial_abstraction PASSED [ 42%]
tests/test_runoff.py::test_impervious_surface PASSED [ 57%]
tests/test_runoff.py::test_negative_rainfall_raises_error PASSED [ 71%]
tests/test_runoff.py::test_invalid_cn_raises_error PASSED [ 85%]
tests/test_runoff.py::test_runoff_never_exceeds_rainfall PASSED [100%]

===== 7 passed in 0.01 seconds =====

```

(a) `python3 -m pytest tests/ -v` — 7 passed.

```

mahmudul-robotics@robotics-ai:~/Downloads/Software Development/ai_water_lab/week5_session_b_lab2$ python3 sensitivity_analysis.py
Saved figures/runoff_sensitivity.png
Check 1: Q <= P for all P, CN - PASS
Check 2: runoff increases with CN at P=50.0 mm - PASS
Q values: [60: 1.4, 70: 5.81, 80: 13.8, 90: 27.11, 95: 36.9]
Check 3: P=80 CN=85 -> Q=43.55 mm (course reference -43.6)

```

(b) `python3 sensitivity_analysis.py` — plot saved; checks PASS.

```

mahmudul-robotics@robotics-ai:~/Downloads/Software Development/ai_water_lab/week5_session_b_lab2$ python3 validate_reference.py
P=50 CN=80 -> Q=13.80 (expect -13.8) OK
P=80 CN=85 -> Q=43.55 (expect -43.55) OK
P=10 CN=80 -> Q=0.00 (expect -0.0) OK
P=50 CN=100 -> Q=50.00 (expect -50.0) OK
mahmudul-robotics@robotics-ai:~/Downloads/Software Development/ai_water_lab/week5_session_b_lab2$

```

(c) `python3 validate_reference.py` — four reference cases OK.

Figure 1: Terminal evidence for Exercises 1–3.

Key numeric outputs from `sensitivity_analysis.py` at $P=50$ mm:

Q values: {60: 1.4, 70: 5.81, 80: 13.8, 90: 27.11, 95: 36.9}
 Check 3: P=80 CN=85 -> Q=43.55 mm (course reference ~43.6)

4.3 Sensitivity figure (Exercise 2)

Figure 2 shows runoff depth vs. rainfall for $CN = 60, 70, 80, 90, 95$. All curves lie at or below the $Q = P$ line; higher CN yields greater runoff for the same P , consistent with more impervious catchments.

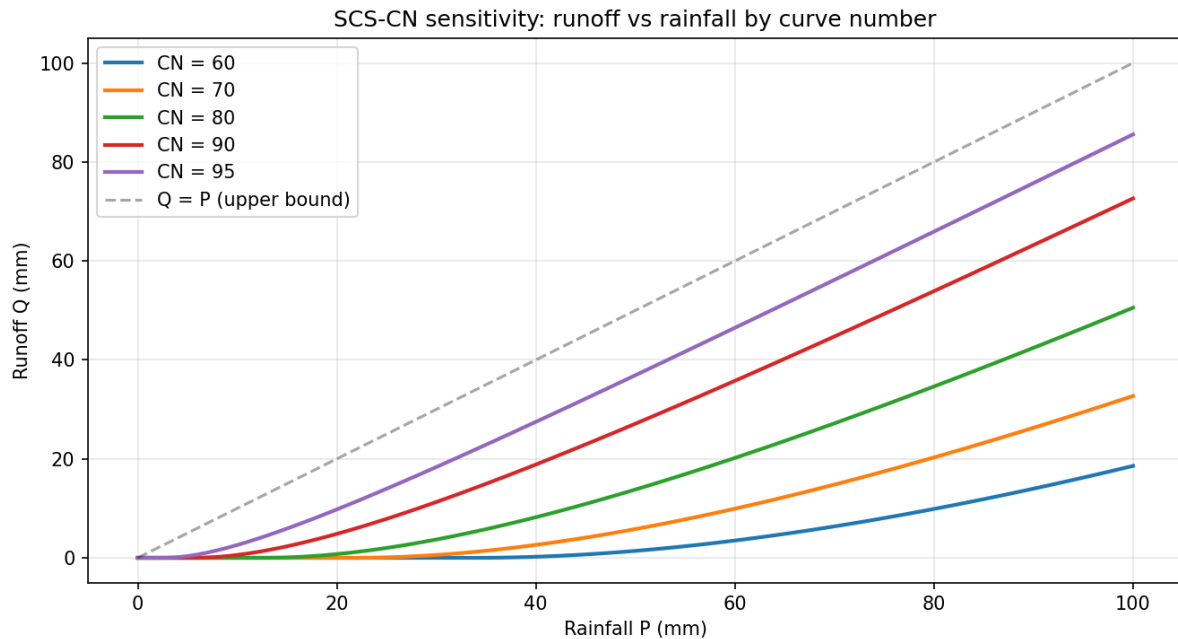


Figure 2: SCS-CN sensitivity: runoff Q (mm) vs. rainfall P (mm) by curve number.

4.4 Validation checklist

Table 2: Exercise 3–4 validation summary.

Check	Outcome
pytest 7/7	PASS (0.01 s)
$Q \leq P$ for all P, CN	PASS
$Q = 0$ when $P < I_a$ (e.g. $P=10, CN=80$)	PASS
Higher CN $\Rightarrow$ higher Q at $P=50$ mm	PASS
$Q(80, 85) \approx 43.55$ mm	Matches course reference
$Q(50, 100) = 50$ mm (impervious)	PASS

5 Analysis and validation

Relation to Week 3 Session B: The same `calculate_runoff` logic was first built under TDD (Red–Green–Refactor). Week 5 Lab 2 adds **sensitivity visualization** and explicit domain scripts rather than new formula changes.

YELLOW/RED rainfall alerts (Week 5 Lab 1): Lab 1 used OpenWeather mm/h thresholds; Lab 2 uses cumulative storm depth P in mm for the SCS-CN equation—different physical quantities, both valid in a Smart Water course.

CN bounds: Minimum CN=1 prevents singular S ; CN=100 gives $S=0$, $I_a=0$, so $Q=P$ for storms exceeding zero depth—a useful impervious limit test.

Optional OpenCode: Prompts `prompt_implement.txt` and `prompt_sensitivity.txt` are available; baseline code and tests were already complete from Week 3, so OpenCode was not required for a passing run.

6 Discussion

1. A single pytest screenshot plus sensitivity and validate screenshots give clear, gradable evidence without rerunning commands.
2. The sensitivity plot makes the CN effect visible; the $Q = P$ reference line reinforces the physical cap enforced in code.
3. Grid test `test_runoff_never_exceeds_rainfall` catches formula slips that hand checks might miss.
4. Live YELLOW/RED wet-weather cases are not needed here; dry API cities in Lab 1 and formula-based Lab 2 cover complementary validation styles.

7 Problems encountered

None blocking: pytest and scripts ran on Python 3.8.10 with pytest 4.6.9. All sampled cities in Lab 1 were dry (GREEN); Lab 2 does not depend on weather API data.

8 Lessons learned

Keeping P , S , I_a , and Q in mm avoids unit bugs. Tests written before or alongside implementation (Week 3 TDD) make sensitivity scripts safe to extend. Document reference pairs such as $(P, CN) = (80, 85)$ in `prompt_log.md` so graders can compare to published examples quickly.

9 Conclusion

Week 5 Session B Lab 2 objectives were met: SCS-CN implementation with tests, sensitivity figure for CN 60–95, and domain validation including course reference runoff ≈ 43.55 mm at $P=80$ mm, CN=85. Submitted: `week5_session_b_lab2/`, `prompt_log.md`, terminal screenshots (Figures 1a–1c), and sensitivity plot (Figure 2).

A Runoff module: `runoff.py`

`src/runoff.py` (full file)

```
from typing import Union

# SCS-CN constants (mm)
_RETENTION_FACTOR = 25400.0
_RETENTION_OFFSET = 254.0
_INITIAL_ABSTRACTION_RATIO = 0.2
_MIN_CN = 1
_MAX_CN = 100

def _validate_positive_rainfall(P: Union[int, float]) -> float:
    """Validate and convert rainfall depth P (mm)."""
    if not isinstance(P, (int, float)):
```

```

        raise TypeError(f"P must be a number, got {type(P).__name__}")
    P = float(P)
    if P < 0:
        raise ValueError(f"Rainfall depth P cannot be negative, got {P}")
    return P

def _validate_curve_number(CN: Union[int, float]) -> float:
    """Validate and convert curve number CN (dimensionless)."""
    if not isinstance(CN, (int, float)):
        raise TypeError(f"CN must be a number, got {type(CN).__name__}")
    CN = float(CN)
    if not (_MIN_CN <= CN <= _MAX_CN):
        raise ValueError(
            f"Curve number CN must be between {_MIN_CN} and {_MAX_CN}, got {CN}"
        )
    return CN

def _compute_runoff(P: float, CN: float) -> float:
    """SCS-CN runoff depth (pre-validated inputs only)."""
    S = _RETENTION_FACTOR / CN - _RETENTION_OFFSET
    Ia = _INITIAL_ABSTRACTION_RATIO * S
    if P <= Ia:
        return 0.0
    effective_rainfall = P - Ia
    return effective_rainfall ** 2 / (effective_rainfall + S)

def calculate_runoff(P: Union[int, float], CN: Union[int, float]) -> float:
    """Compute direct runoff Q (mm) using the SCS-CN method.

    The Soil Conservation Service Curve Number (SCS-CN) method estimates
    rainfall excess (direct runoff) from total rainfall and a curve number.

    Parameters
    -----
    P : float
        Total rainfall depth (mm). Must be >= 0.
    CN : float
        Curve number in [1, 100]. dimensionless.

    Returns
    -----
    float
        Direct runoff depth Q (mm), clipped so that 0 <= Q <= P.

    Raises
    -----
    ValueError
        If P < 0 or CN is outside [1, 100].

    Formulas
    -----
    S = 25400 / CN - 254                (maximum retention, mm)
    Ia = 0.2 * S                        (initial abstraction, mm)
    If P <= Ia: Q = 0
    Else:      Q = (P - Ia)**2 / (P - Ia + S)
    """
    P = _validate_positive_rainfall(P)
    CN = _validate_curve_number(CN)
    Q = _compute_runoff(P, CN)
    return max(0.0, min(Q, P))

```

B Tests: test_runoff.py

tests/test_runoff.py (full file)

```
import pytest
from src.runoff import calculate_runoff

def test_normal_case():
    """P=50 mm, CN=80 → hand-calculated Q 13.81 mm."""
    P = 50.0
    CN = 80
    # S = 25400/80 - 254 = 63.5 mm
    # Ia = 0.2 * 63.5 = 12.7 mm
    # Q = (50 - 12.7)**2 / (50 - 12.7 + 63.5)
    #   = 37.3**2 / 100.8
    #   = 1391.29 / 100.8 13.8025
    expected = 13.80
    result = calculate_runoff(P, CN)
    assert result == pytest.approx(expected, abs=0.01)

def test_no_rainfall():
    """P=0 → Q must be 0 regardless of CN."""
    assert calculate_runoff(0.0, 80) == 0.0
    assert calculate_runoff(0.0, 50) == 0.0
    assert calculate_runoff(0.0, 100) == 0.0

def test_rainfall_less_than_initial_abstraction():
    """P < Ia → no runoff (Q = 0)."""
    # CN=80 → Ia=12.7 mm; P=10 < 12.7
    assert calculate_runoff(10.0, 80) == 0.0

def test_impervious_surface():
    """CN=100 → S=0, Ia=0, Q=P."""
    P = 50.0
    result = calculate_runoff(P, 100)
    assert result == pytest.approx(P, abs=1e-9)

def test_negative_rainfall_raises_error():
    """Negative P is physically impossible → ValueError."""
    with pytest.raises(ValueError):
        calculate_runoff(-1.0, 80)
    with pytest.raises(ValueError):
        calculate_runoff(-0.1, 80)

def test_invalid_cn_raises_error():
    """CN outside [1, 100] → ValueError."""
    with pytest.raises(ValueError):
        calculate_runoff(50.0, 0)
    with pytest.raises(ValueError):
        calculate_runoff(50.0, 101)
    with pytest.raises(ValueError):
        calculate_runoff(50.0, -1)

def test_runoff_never_exceeds_rainfall():
    """Physical validation: Q ≤ P for a range of inputs."""
    import itertools
```

```

for P, CN in itertools.product([0, 5, 10, 25, 50, 100, 200],
                               [1, 30, 50, 70, 80, 90, 100]):
    q = calculate_runoff(float(P), CN)
    assert 0.0 <= q <= float(P), f"Q={q} out of [0, {P}] for CN={CN}"

```

C Sensitivity script: sensitivity_analysis.py

sensitivity_analysis.py (full file)

```

#!/usr/bin/env python3
"""Exercise 2: SCS-CN parameter sensitivity | rainfall vs runoff by CN."""

from __future__ import annotations

import os

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from src.runoff import calculate_runoff

CN_VALUES = [60, 70, 80, 90, 95]
P_MIN, P_MAX, P_STEP = 0, 100, 1 # mm

def build_sensitivity_dataframe() -> pd.DataFrame:
    """Wide format: rainfall_mm, Q_CN60, Q_CN70, ..."""
    rainfall = np.arange(P_MIN, P_MAX + P_STEP, P_STEP, dtype=float)
    data = {"rainfall_mm": rainfall}
    for cn in CN_VALUES:
        data[f"Q_CN{cn}"] = [calculate_runoff(float(p), cn) for p in rainfall]
    return pd.DataFrame(data)

def plot_sensitivity(df: pd.DataFrame, out_path: str) -> None:
    fig, ax = plt.subplots(figsize=(9, 5))
    for cn in CN_VALUES:
        ax.plot(
            df["rainfall_mm"],
            df[f"Q_CN{cn}"],
            label=f"CN = {cn}",
            linewidth=2,
        )
    ax.plot(
        df["rainfall_mm"],
        df["rainfall_mm"],
        "k--",
        alpha=0.35,
        label="Q = P (upper bound)",
    )
    ax.set_xlabel("Rainfall P (mm)")
    ax.set_ylabel("Runoff Q (mm)")
    ax.set_title("SCS-CN sensitivity: runoff vs rainfall by curve number")
    ax.legend()
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    os.makedirs(os.path.dirname(out_path) or ".", exist_ok=True)
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    print(f"Saved {out_path}")

```

```

def domain_checks(df: pd.DataFrame) -> None:
    """Exercise 3: quick physical validation prints."""
    for cn in CN_VALUES:
        col = f"Q_CN{cn}"
        q = df[col].values
        p = df["rainfall_mm"].values
        assert np.all(q <= p + 1e-9), f"Q > P for CN={cn}"
    print("Check 1: Q <= P for all P, CN | PASS")
    # Higher CN -> more runoff for same P (where P > Ia for all)
    p_test = 50.0
    row = df.loc[df["rainfall_mm"] == p_test].iloc[0]
    qs = [row[f"Q_CN{cn}"] for cn in CN_VALUES]
    assert qs == sorted(qs), "Higher CN should yield >= runoff at P=50"
    print(f"Check 2: runoff increases with CN at P={p_test} mm | PASS")
    print(f"  Q values: {dict(zip(CN_VALUES, [round(q, 2) for q in qs]))}")
    # Hand check Week 1/3 reference
    ref = calculate_runoff(80, 85)
    print(f"Check 3: P=80 CN=85 -> Q={ref:.2f} mm (course reference ~43.6)")

def main() -> None:
    df = build_sensitivity_dataframe()
    df.to_csv("figures/sensitivity_data.csv", index=False)
    plot_sensitivity(df, "figures/runoff_sensitivity.png")
    domain_checks(df)

if __name__ == "__main__":
    main()

```

D Reference checks: validate_reference.py

validate_reference.py (full file)

```

#!/usr/bin/env python3
"""Spot-check reference cases for prompt_log / Exercise 3."""

from src.runoff import calculate_runoff

CASES = [
    (50, 80, 13.80),
    (80, 85, 43.55),
    (10, 80, 0.0),
    (50, 100, 50.0),
]

if __name__ == "__main__":
    for p, cn, expected in CASES:
        q = calculate_runoff(p, cn)
        ok = abs(q - expected) < 0.1 or (expected == 0 and q == 0)
        print(f"P={p} CN={cn} -> Q={q:.2f} (expect {expected}) {'OK' if ok else  

        ↪ 'CHECK'}")

```


E Submitted prompt log: prompt_log.md

prompt_log.md (full file)

```
# Prompt Log - Week 5 Session B Lab 2

**Student:** Mahmudul Hasan (4125999049)
**Lab:** SCS-CN Runoff Modeling

---

## Implementation

**CN validation rule used:** CN in [1, 100] (minimum 1 avoids divide-by-zero on  $S = \frac{25400}{CN} - 254$ )
↪ 25400/CN - 254)

**Trickiest edge case:** P just below  $I_a$  returns 0; CN=100 gives  $Q=P$  for large P
↪ (impervious limit)

**Prompt used (Exercise 1):** `prompt_implement.txt` (code reused from Week 3 Session B
↪ TDD lab)

**pytest result:** 7 passed (`python3 -m pytest tests/ -v`)

**Hand check:** P=50 CN=80 → Q13.80 mm; P=80 CN=85 → Q43.55 mm

---

## Sensitivity

**Figure path:** `figures/runoff_sensitivity.png`

**Data:** `figures/sensitivity_data.csv`

**CN values:** 60, 70, 80, 90, 95

**Main trend in one sentence:** For the same rainfall, higher curve number (more
↪ impervious land) produces more runoff; all curves rise with P and stay below the  $Q=P$ 
↪ line.

**Prompt used (Exercise 2):** `prompt_sensitivity.txt`

---

## Domain validation

**Published reference used:** Course Week 1 Session B / Week 3 labs (P=80 mm, CN=85
↪ Q43.6 mm)

**Checks run:** QP; Q=0 when  $P < I_a$ ; higher CN → higher Q at P=50 mm

**Any mismatch and resolution:** None | P=80, CN=85 matches course reference (~43.55 mm)

---

## Lessons learned

[Your own words]
```